

```

1 //SOLUCIONARIO LAB2 2026-2
2
3 import java.util.ArrayList;
4 import java.util.Date;
5 import java.util.List;
6
7 // Clase de arranque. Aqui NO hay logica de negocio: solo se arma el
8 // escenario de prueba (preguntas, profesores, examen) y se manda a mostrar.
9 public class Main {
10     public static void main(String[] args) {
11
12         // 1. Banco de preguntas
13         // QuestionManager es el "almacen": guarda todas las preguntas
14         // disponibles del curso, no las del examen todavia.
15         QuestionManager questionManager = new QuestionManager();
16
17         // Dos tipos distintos de pregunta, ambos hijos de Question.
18         // SingleChoice guarda UN indice de respuesta correcta (int).
19         // MultipleChoice guarda VARIOS (List<Integer>). Esa es toda la
20         // diferencia de estado entre las dos subclases.
21         SingleChoiceQuestion sc1 = new SingleChoiceQuestion(1, "1 + 1?", List.of("1", "2", "11", "0"), 2);
22         SingleChoiceQuestion sc2 = new SingleChoiceQuestion(2, "2*4?", List.of("24", "2", "8", "16"), 3);
23         MultipleChoiceQuestion mc1 = new MultipleChoiceQuestion(3, "is a planet?", List.of("Earth", "Moon", "E
24         urope", "Mars"), List.of(1, 4));
25
26         // add() recibe Question, no SingleChoiceQuestion: por polimorfismo
27         // cualquier hija entra por el mismo metodo.
28         questionManager.add(sc1);
29         questionManager.add(sc2);
30         questionManager.add(mc1);
31
32         // 2. Registro de Profesores
33         Teacher teacher1 = new Teacher("11111111", "Freddy", "Paz");
34         Teacher teacher2 = new Teacher("22222222", "Jose", "Corcuera");
35
36         List<Teacher> teachers = new ArrayList<>();
37         teachers.add(teacher1);
38         teachers.add(teacher2);
39
40         // 3. Seleccion de preguntas para el examen y asignacion de puntajes
41         // El manager devuelve 3 preguntas al azar y sin repetir.
42         List<Question> questionsSelected = questionManager.selectRandomN(3);
43
44         // Cada pregunta seleccionada se envuelve en un QuestionAssessment,
45         // que le pega el puntaje que vale DENTRO de este examen.
46         // El puntaje no vive en Question porque la misma pregunta puede
47         // valer distinto en dos exámenes diferentes.
48         List<QuestionAssessment> puntajeEnPreguntas = new ArrayList<>();
49         QuestionAssessment aux;
50         int puntaje = 5;
51         for (Question questionSelect : questionsSelected) {
52             aux = new QuestionAssessment(questionSelect, puntaje);
53             puntajeEnPreguntas.add(aux);
54         }
55
56         // 4. Creacion del examen
57         // new Date() = fecha y hora de este instante.
58         Assessment assessment = new Assessment(120, new Date(), teachers, puntajeEnPreguntas);
59
60         // El examen NO se imprime a si mismo: de eso se encarga el viewer.
61         // Separar datos de presentacion permite cambiar la salida (consola,
62         // archivo, web) sin tocar la clase Assessment.
63         AssessmentViewerConsole.show(assessment);
64     }
65 }
66
67 import java.util.List;
68
69 // Pregunta de opcion unica: una sola alternativa es correcta.
70 // 'extends Question' le da gratis prompt, codigo y sus getters.
71 class SingleChoiceQuestion extends Question {
72     private List<String> opciones;
73     private int respuesta; // indice de la unica alternativa correcta
74
75     public SingleChoiceQuestion(int codigo, String prompt, List<String> opciones, int respuesta) {
76         // super() llama al constructor del padre y va SIEMPRE en la 1ra linea.
77         super(codigo, prompt);
78         this.opciones = opciones;
79         this.respuesta = respuesta;
80     }
81
82     // Implementacion propia del metodo abstracto heredado.
83     public String mostrarPregunta() {

```

```

82     String cadena = "";
83     cadena += "Pregunta " + getCodigo() + " " + getPrompt() + "\n";
84     String aux = "";
85     for (String opc : opciones) {
86         // [DEFECTO del solucionario] imprime "1." en todas las opciones.
87         // Deberia haber un contador que incremente en cada vuelta.
88         aux += "1. " + opc + "\n";
89     }
90     aux += "Ingresa su respuesta: " + "\n";
91     cadena += aux;
92     return cadena;
93 }
94 }
95 import java.util.List;
96
97 // Pregunta de opcion multiple: varias alternativas correctas.
98 // Unica diferencia real con SingleChoice: guarda una LISTA de respuestas
99 // en vez de un solo int. El resto del sistema no nota la diferencia.
100 class MultipleChoiceQuestion extends Question {
101     private List<String> opciones;
102     private List<Integer> respuestas;
103
104     public MultipleChoiceQuestion(int codigo, String prompt,
105         List<String> opciones, List<Integer> respuestas) {
106         super(codigo, prompt);
107         this.opciones = opciones;
108         this.respuestas = respuestas;
109     }
110
111     public String mostrarPregunta() {
112         String cadena = "";
113         cadena += "Pregunta " + getCodigo() + " " + getPrompt() + "\n";
114         String aux = "";
115         for (String opc : opciones) {
116             // [DEFECTO del solucionario] mismo bug que en SingleChoice.
117             aux += "1. " + opc + "\n";
118         }
119         // Lo unico que cambia en pantalla: el texto pide varias opciones.
120         aux += "Seleccione las opciones de su respuesta: " + "\n";
121         cadena += aux;
122         return cadena;
123     }
124 }
125 // Clase base de toda pregunta.
126 // ABSTRACTA porque "una pregunta generica" no existe: siempre es de
127 // opcion unica o multiple. No se puede hacer new Question(...).
128 // Guarda lo comun a cualquier tipo: enunciado y codigo.
129 abstract class Question implements Printable {
130     // private: el estado queda encerrado. Las hijas acceden por getters.
131     private String prompt;
132     private int codigo;
133
134     // Metodo abstracto: obliga a cada hija a dar su propia version.
135     // Es lo que hace posible el polimorfismo mas abajo.
136     public abstract String mostrarPregunta();
137
138     public Question(int codigo, String prompt) {
139         this.prompt = prompt;
140         this.codigo = codigo;
141     }
142
143     public String getPrompt() {
144         return prompt;
145     }
146
147     public int getCodigo() {
148         return codigo;
149     }
150 }
151 import java.util.ArrayList;
152 import java.util.List;
153 import java.util.Random;
154
155 // Administra el banco de preguntas del curso.
156 // Su unica responsabilidad es guardar preguntas y prestar N al azar.
157 class QuestionManager {
158     // Lista del tipo del PADRE: acepta cualquier hija de Question.
159     private List<Question> preguntas;
160
161     public QuestionManager() {
162         // Inicializar la lista en el constructor evita NullPointerException
163         // al primer add(). Una lista declarada y no creada vale null.

```

```

164     preguntas = new ArrayList<>();
165 }
166
167 public void add(Question dato) {
168     this.preguntas.add(dato);
169 }
170
171 public List<Question> selectRandomN(int n) {
172     // Copiar las preguntas en otra lista para no borrar las disponibles.
173     // Se trabaja sobre 'aux' y el banco original queda intacto.
174     List<Question> aux = new ArrayList<>();
175     List<Question> selecciones = new ArrayList<>();
176     for (Question q : preguntas) {
177         aux.add(q);
178     }
179
180     // Selecciona al azar n preguntas.
181     Random rand = new Random();
182     int indice;
183     for (int i = 1; i <= n; i++) {
184         // nextInt(tope) devuelve de 0 a tope-1: justo el rango de indices.
185         indice = rand.nextInt(aux.size());
186         selecciones.add(aux.get(indice));
187         // Al removerla de 'aux' ya no puede volver a salir: asi se
188         // garantiza que no se repitan preguntas en el mismo examen.
189         aux.remove(indice);
190     }
191     return selecciones;
192 }
193 }
194 // Profesor a cargo del examen. Clase de puro dato (sin comportamiento).
195 class Teacher {
196     private String codigo;
197     private String nombre;
198     private String apellido;
199
200     public Teacher(String codigo, String nombre, String apellido) {
201         this.codigo = codigo;
202         this.nombre = nombre;
203         this.apellido = apellido;
204     }
205 }
206 import java.util.Date;
207 import java.util.List;
208
209 // El examen concreto: cuanto dura, cuando empieza, quien lo dicta
210 // y que preguntas (con su puntaje) lo componen.
211 class Assessment {
212     private int duracion;
213     private Date fecha;
214     private List<Teacher> profesores;
215     // Ojo: NO es List<Question>. Es List<QuestionAssessment> porque aqui
216     // cada pregunta ya viene con el puntaje que vale en este examen.
217     private List<QuestionAssessment> preguntas;
218
219     public Assessment(int duracion, Date fecha, List<Teacher> profesores, List<QuestionAssessment> preguntas)
220     {
221         this.duracion = duracion;
222         this.profesores = profesores;
223         this.fecha = fecha;
224         this.preguntas = preguntas;
225     }
226
227     public int getDuracion() {
228         return duracion;
229     }
230
231     public Date getFecha() {
232         return fecha;
233     }
234
235     // Recorre las preguntas y va concatenando el texto de cada una.
236     // Aqui ocurre el POLIMORFISMO: mostrarPregunta() se resuelve en
237     // tiempo de ejecucion segun si el objeto es Single o Multiple choice.
238     public String mostrarPreguntasEnExamen() {
239         String cadena = "";
240         for (QuestionAssessment pregunta : preguntas) {
241             cadena += pregunta.getPregunta().mostrarPregunta();
242         }
243         return cadena;
244     }
245 }

```

```

245 // Clase de union: relaciona una pregunta con el puntaje que vale
246 // dentro de un examen especifico.
247 // Existe porque el puntaje no es propiedad de la pregunta: la misma
248 // pregunta puede valer 5 en un examen y 2 en otro.
249 class QuestionAssessment {
250     private Question pregunta;
251     private int puntaje;
252
253     public QuestionAssessment(Question pregunta, int puntaje) {
254         this.pregunta = pregunta;
255         this.puntaje = puntaje;
256     }
257
258     public Question getPregunta() {
259         return pregunta;
260     }
261
262     public int getPuntaje() {
263         return puntaje;
264     }
265 }
266 import java.text.SimpleDateFormat;
267 import java.util.Date;
268
269 // Unica responsabilidad: mostrar un examen por consola.
270 // Separarlo de Assessment permite tener otros viewers (archivo, web)
271 // sin tocar la clase que guarda los datos.
272 class AssessmentViewerConsole {
273     // static: se llama como AssessmentViewerConsole.show(...), sin
274     // necesidad de crear un objeto viewer.
275     public static void show(Assessment examen) {
276         System.out.println("Inicio de examen =====");
277         System.out.println("Duracion: " + examen.getDuracion() + " minutos");
278         // SimpleDateFormat convierte el Date (que por dentro es un numero)
279         // en texto legible con el patron indicado.
280         SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy HH:mm");
281         System.out.println("Fecha hora de inicio : " + sdf.format(examen.getFecha()));
282         System.out.print(examen.mostrarPreguntasEnExamen());
283     }
284 }
285 // Contrato: cualquier cosa "imprimible" sabe devolver su texto.
286 // Una interfaz solo declara la firma; no dice COMO se arma ese texto.
287 public interface Printable {
288     public String mostrarPregunta();
289 }

```